

文章笔记

Building Effective Agents — 构建高效 Agent 的实践指南

基于 Anthropic 官方博客整理

2026 年 4 月 2 日

作者: Erik Schluntz, Barry Zhang (Anthropic)

发布日期: 2024-12-19

预计阅读时长: 约 20 分钟阅读

文章链接: <https://www.anthropic.com/engineering/building-effective-agents>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：为什么要读这篇文章	2
1.1 本章小结	2
2 基本概念：Agentic 系统的分类	2
2.1 什么是 Agent?	2
2.2 何时使用（以及何时不使用）Agentic 系统	2
2.3 框架的使用建议	3
2.4 本章小结	3
3 基础构件：增强型 LLM	3
3.1 本章小结	4
4 五种 Workflow 模式详解	4
4.1 模式一：Prompt Chaining（提示链）	4
4.2 模式二：Routing（路由）	5
4.3 模式三：Parallelization（并行化）	5
4.4 模式四：Orchestrator-Workers（编排者-工作者）	6
4.5 模式五：Evaluator-Optimizer（评估者-优化者）	6
4.6 五种模式对比总览	7
4.7 本章小结	7
5 自主 Agent 的设计	7
5.1 Agent 的定义与工作机制	7
5.2 何时使用 Agent	7
5.3 Agent 的实践案例	8
5.4 本章小结	8
6 Agent 在实践中的应用领域	8
6.1 客户支持（Customer Support）	8
6.2 编程 Agent（Coding Agents）	9
6.3 本章小结	9
7 工具设计的最佳实践	9
7.1 Agent-Computer Interface (ACI)	9
7.2 工具格式的选择	10
7.3 本章小结	10
8 总结与延伸	11
8.1 三大核心原则	11
8.2 从本文可以学到什么	11
8.3 拓展阅读	11

1 引言：为什么要读这篇文章

Anthropic 是 Claude 系列大语言模型的开发商，在过去一年中与数十个行业团队合作构建基于 LLM 的 Agent 系统。这篇文章凝聚了他们的第一手实践经验，核心发现是：**最成功的实现往往不依赖复杂框架，而是采用简单、可组合的设计模式。**

本文的独特价值在于：它不是一篇学术论文，而是来自工业界一线的「设计模式手册」。文章系统性地将 Agentic 系统分解为 **Workflow**（工作流）和 **Agent**（自主代理）两大类，详细介绍了五种 Workflow 模式和 Agent 的设计原则，并在附录中给出了实践案例和工具设计的最佳实践。

本文核心论点

构建 LLM 应用的成功之道不在于打造最复杂的系统，而在于为需求打造**最合适**的系统。应从简单 prompt 出发，通过系统评估优化，仅在简单方案不足时才引入多步骤 agentic 系统。

1.1 本章小结

本文来自 Anthropic 的工程实践总结，核心主张是「从简到繁」地构建 agentic 系统。理解这篇文章有助于在实际开发中做出正确的架构决策。

2 基本概念：Agentic 系统的分类

2.1 什么是 Agent？

「Agent」一词在业界有多种定义：

- **广义**：能独立运行较长时间、使用多种工具完成复杂任务的全自主系统。
- **狭义**：按预定义工作流执行的程序化实现。

Anthropic 将所有这些变体统称为 **Agentic Systems**（代理系统），但在架构层面做出了一个关键区分：

Workflow vs. Agent 的核心区分

- **Workflow**（工作流）：LLM 和工具通过**预定义的代码路径**进行编排。控制流由开发者决定。
- **Agent**（自主代理）：LLM **动态地自主决定**其处理流程和工具使用。控制权交给模型本身。

这一区分的意义在于：Workflow 更可预测、更可控，适合结构化任务；Agent 更灵活，适合开放性问题。两者并非对立，而是复杂度谱系上的不同位置。

2.2 何时使用（以及何时不使用）Agentic 系统

Anthropic 给出了一个明确的决策框架：

复杂度递增的决策路径

1. **首先考虑最简单的方案**：单次 LLM 调用 + Retrieval + In-context Examples 是否已经足够？
2. **需要更高可靠性**：使用 Workflow 模式，获得可预测性和一致性。
3. **需要灵活性和模型驱动决策**：使用 Agent 模式。

过早引入复杂度的陷阱

Agentic 系统通常以**更高的延迟和成本**换取更好的任务表现。在项目初期就引入复杂的 Agent 架构是常见的错误——很多场景下，经过精心优化的单次 LLM 调用完全可以胜任。一定要先评估简单方案的上限，再决定是否引入更复杂的模式。

2.3 框架的使用建议

市面上有很多简化 agentic 系统开发的框架，如 Claude Agent SDK、AWS Strands Agents SDK、Rivet（拖拽式 GUI 工作流构建器）等。Anthropic 对框架的建议是：

- **先用原生 API 起步**：很多模式只需几行代码即可实现。
- **理解底层实现**：框架的抽象层可能遮蔽 prompt 和 response 的细节，增加 debug 难度。
- **避免不必要的复杂度**：框架容易诱导开发者在简单方案足够时添加不必要的复杂性。

框架使用的常见错误

对底层实现的**错误假设**是客户出错的常见原因。使用框架时，务必理解框架在底层做了什么——它发送了什么 prompt？如何解析 tool call？中间步骤的结果如何传递？不理解这些就无法有效 debug。

2.4 本章小结

Agentic 系统分为 Workflow 和 Agent 两大类。Workflow 由代码控制流程，Agent 由模型控制流程。开发时应从简单方案出发，仅在必要时增加复杂度。框架可以帮助快速起步，但要理解底层实现。

3 基础构件：增强型 LLM

在介绍具体的 Workflow 模式之前，需要先理解 agentic 系统的基础构件——**Augmented LLM**（增强型 LLM）。

增强型 LLM 是在基础 LLM 之上叠加了三种核心能力的模块：

增强能力	说明
Retrieval (检索)	从外部知识库、数据库或文档中获取相关信息，扩展模型的知识边界
Tools (工具)	调用外部 API、执行代码、操作文件等，使模型能够影响外部世界
Memory (记忆)	在多轮交互中保持上下文，记录和回忆之前的信息

表 1: 增强型 LLM 的三大核心能力

当前的模型已能**主动**使用这些能力——自行生成搜索查询、选择合适的工具、决定保留哪些信息。Anthropic 建议在实现时关注两个方面：

1. **针对具体用例定制**这些能力，而非追求通用性。
2. 确保这些能力为 LLM 提供**简洁、文档完备的接口**。

Model Context Protocol (MCP)

Anthropic 推出的 MCP 协议允许开发者通过统一的客户端实现接入不断增长的第三方工具生态。这意味着增强型 LLM 的工具层可以标准化、可复用，而非每个项目重新开发。MCP 正成为 Agent 工具生态的重要基础设施。

3.1 本章小结

增强型 LLM 是所有 agentic 系统的基础模块，由 Retrieval、Tools、Memory 三种能力增强。后续所有 Workflow 和 Agent 模式都假设每次 LLM 调用都具备这些增强能力。

4 五种 Workflow 模式详解

本节是文章的核心内容，详细介绍了五种在生产环境中常见的 Workflow 设计模式。

4.1 模式一：Prompt Chaining (提示链)

核心思想：将任务分解为一系列**顺序执行**的步骤，每个 LLM 调用处理前一个调用的输出。可以在中间步骤加入程序化检查 (Gate) 以确保流程正确。

适用场景：任务可以清晰地分解为固定的子任务，通过将每个 LLM 调用变为更简单的任务来**用延迟换取更高准确率**。

典型案例：

- 先生成营销文案，再翻译成其他语言。
- 先写文档大纲，检查大纲是否满足标准，再基于大纲撰写全文。

Prompt Chaining 的设计哲学

Prompt Chaining 的核心权衡是：**用更多的 LLM 调用次数和更高的延迟，换取每个步骤更高的准确性**。它之所以有效，是因为将一个复杂任务拆分为多个简单任务后，每个子任务对 LLM 来说更容易完成。中间的 Gate 检查机制是保障质量的关键——如果中间步骤出错，可以提前中止而非让错误传播下去。

4.2 模式二：Routing（路由）

核心思想：对输入进行**分类**，然后将其导向专门的后续处理流程。这一模式实现了关注点分离，使得每个分支可以使用更专门的 prompt。

适用场景：复杂任务存在多个不同类别，每个类别需要不同的处理方式，且分类可以被准确执行。

典型案例：

- 将客服查询按类型（一般咨询、退款请求、技术支持）路由到不同的处理流程。
- 将简单问题路由到小型高效模型（如 Claude Haiku 4.5），将复杂问题路由到能力更强的模型（如 Claude Sonnet 4.5）。

Routing 的深层价值

Routing 的价值不仅在于效率优化，更在于避免「一个 prompt 打天下」的困境。在实践中，为一种输入优化 prompt 往往会**损害**对其他类型输入的表现。通过 Routing 将不同类型的输入分流到各自专门的处理路径，可以实现整体性能的帕累托改进。

4.3 模式三：Parallelization（并行化）

核心思想：多个 LLM 同时处理任务，结果通过程序化方式聚合。有两种变体：

变体	说明
Sectioning（分段）	将任务拆分为 相互独立 的子任务，并行执行以提高速度
Voting（投票）	对 同一任务 运行多次，获取多样化的输出以提高置信度

表 2: Parallelization 的两种变体

Sectioning 案例：

- 一个 LLM 实例处理用户查询，另一个实例并行进行内容安全审查（Guardrails）。将安全检查与核心任务分离，通常比让同一个 LLM 同时处理两者效果更好。
- 自动化评估中，每个 LLM 调用评估模型表现的不同方面。

Voting 案例：

- 多个不同 prompt 并行审查代码漏洞，只要有一个发现问题就标记。
- 多个 prompt 从不同角度评估内容是否合规，通过投票阈值平衡误报和漏报。

Parallelization 的适用前提

并行化要求子任务之间**相互独立**（Sectioning）或任务本身具有随机性（Voting）。如果子任务之间存在依赖关系，强行并行化会导致信息丢失和结果质量下降。此外，并行化会线性增加 API 调用成本，需要在速度/质量和成本之间做好权衡。

4.4 模式四：Orchestrator-Workers（编排者-工作者）

核心思想：一个中央 LLM（Orchestrator）**动态地**分解任务，将子任务委派给 Worker LLM，最后综合各 Worker 的结果。

与 Parallelization 的区别：虽然拓扑结构相似，关键区别在于**灵活性**——Parallelization 的子任务是预定义的，而 Orchestrator-Workers 的子任务由 Orchestrator 根据具体输入动态决定。

适用场景：无法预先确定需要哪些子任务的复杂任务。

典型案例：

- 编程产品中，每次需要对多个文件进行复杂修改（文件数量和修改内容取决于任务）。
- 需要从多个来源搜集和分析信息的搜索任务。

Orchestrator-Workers 的核心价值

这一模式的核心价值在于处理「**事先不知道要做什么**」的场景。与预定义分支的 Parallelization 不同，Orchestrator 可以根据输入的具体内容灵活决定需要哪些 Worker、分配什么任务。这使其特别适合像代码重构这样子任务数量和内容都不确定的场景。

4.5 模式五：Evaluator-Optimizer（评估者-优化者）

核心思想：一个 LLM 生成响应，另一个 LLM 提供评估和反馈，形成**迭代优化循环**。这类似于人类作家反复修改打磨文章的过程。

适用前提（两个信号）：

1. 人类给出反馈时，LLM 的输出可以**明显改善**。
2. LLM 本身能够**提供**这样的反馈。

典型案例：

- 文学翻译：翻译 LLM 可能遗漏某些细微差别，但评估 LLM 可以给出有价值的批评。
- 复杂搜索任务：需要多轮搜索和分析才能收集全面信息，由评估者决定是否需要继续搜索。

Evaluator-Optimizer 与强化学习的类比

这一模式在概念上与强化学习中的 Actor-Critic 架构有相似之处：Generator 类似 Actor（执行动作），Evaluator 类似 Critic（评估价值）。迭代循环使得系统能够逐步逼近最优输出，而无需一次性生成完美结果。判断是否适合使用此模式的关键是：评估输出的质量是否比生成高质量输出更容易。

4.6 五种模式对比总览

模式	控制流特征	适用场景	核心权衡
Prompt Chain-ing	串行，固定步骤	可清晰分解的任务	延迟换准确率
Routing	分类后分流	多类别输入	分类准确性要求
Parallelization	并行，预定义分支	独立子任务/多视角	成本换速度/置信度
Orchestrator-Workers	动态分解委派	子任务不可预知	灵活性换可控性
Evaluator-Optimizer	迭代反馈循环	有明确评估标准	延迟换质量

表 3: 五种 Workflow 模式对比

4.7 本章小结

五种 Workflow 模式覆盖了从简单串行到复杂迭代的多种场景。选择哪种模式取决于任务的结构特征：子任务是否固定、是否独立、是否可预知、是否有明确的评估标准。在实践中，这些模式可以自由组合和定制。

5 自主 Agent 的设计

5.1 Agent 的定义与工作机制

当 LLM 在关键能力上日趋成熟——理解复杂输入、进行推理与规划、可靠地使用工具、从错误中恢复——Agent 开始在生产环境中出现。

Agent 的工作流程如下：

1. 从人类用户的命令或交互讨论开始。
2. 任务明确后，Agent **独立**进行规划和操作。
3. 在执行过程中，通过工具调用结果、代码执行等从环境获取「**Ground Truth**」来评估进展。
4. 可以在检查点（Checkpoint）暂停，请求人类反馈。
5. 任务完成时终止，也通常设有最大迭代次数等停止条件。

Agent 的实现本质

Agent 听起来复杂，但其实现往往非常直接：**本质上就是 LLM 在一个循环中基于环境反馈使用工具**。关键不在于复杂的架构，而在于精心设计的工具集和清晰的工具文档。工具的设计质量决定了 Agent 的上限。

5.2 何时使用 Agent

Agent 适用于以下场景：

- **开放性问题**：无法预测所需步骤数。

- 无法硬编码固定路径：需要模型自主决策。
- 可信环境：对模型的决策有一定信任度。
- 规模化需求：需要在受信任环境中大规模执行任务。

Agent 的风险与代价

Agent 的自主性意味着：(1) **更高的成本**——多轮工具调用的 token 消耗远超单次调用；(2) **错误可能复合**——一个步骤的错误可能在后续步骤中放大。Anthropic 建议在**沙箱环境**中进行充分测试，并设置适当的 Guardrails（如最大迭代次数、敏感操作需人工确认等）。

5.3 Agent 的实践案例

Anthropic 自身的两个 Agent 实现：

案例	说明
SWE-bench Cod-ing Agent	基于 Pull Request 描述，自动编辑多个文件以解决真实 GitHub Issue。能在 SWE-bench Verified 基准测试中解决实际问题。
Computer Use Agent	Claude 使用计算机（鼠标、键盘、屏幕）来完成各种任务，是多模态 Agent 的典型示例。

表 4: Anthropic 的 Agent 实践案例

5.4 本章小结

Agent 是 Agentic 系统复杂度谱系的最高端，适合开放性、不可预知路径的任务。其实现本质是 LLM + 工具调用循环，关键在于工具设计而非架构复杂度。使用时需注意成本、错误复合风险，以及充分的沙箱测试。

6 Agent 在实践中的应用领域

文章附录详细讨论了两个最有前景的 Agent 应用领域，它们共享以下特征：需要对话与行动的结合、有明确的成功标准、能形成反馈循环、并整合有意义的人类监督。

6.1 客户支持 (Customer Support)

客户支持是 Agent 的天然契合场景：

- 支持交互本身就是对话流，同时需要访问外部信息和执行操作。
- 可以集成工具来拉取客户数据、订单历史、知识库文章。
- 退款、更新工单等操作可以程序化处理。
- 成功可以通过用户确认的解决方案来明确衡量。

商业模式验证

多家公司已通过**基于成功解决方案收费**（usage-based pricing）的商业模式验证了这一方向——只有当 Agent 成功解决用户问题时才收费。这种商业模式本身就是对 Agent 能力的最强信号。

6.2 编程 Agent (Coding Agents)

软件开发领域展现了 LLM Agent 的巨大潜力，能力从代码补全进化到自主问题解决。编程 Agent 之所以特别有效，原因在于：

- 代码解决方案可以通过**自动化测试**验证。
- Agent 可以用测试结果作为反馈进行迭代。
- 问题空间定义明确、结构化。
- 输出质量可以客观衡量。

自动化测试不等于完全验证

虽然自动化测试能验证代码的功能正确性，但**人类审查**仍然不可或缺——需要确保解决方案符合更广泛的系统需求、代码风格规范、性能要求、安全标准等。测试通过只是最低门槛，不是质量保证的全部。

6.3 本章小结

客户支持和编程是目前最成熟的 Agent 应用领域。前者契合于对话式交互 + 操作的模式，后者受益于代码的可测试性和问题空间的结构化。两者都需要人类监督来保证更高层次的质量。

7 工具设计的最佳实践

文章的附录二是一个极具实操价值的章节，讨论了如何为 Agent 设计工具接口——这是决定 Agent 效果的关键因素。

7.1 Agent-Computer Interface (ACI)

Anthropic 提出了一个重要概念：**ACI** (Agent-Computer Interface)，与 HCI (Human-Computer Interface) 相对应。核心观点是：设计 Agent 工具接口时，应该投入**与设计人机界面同等**的精力。

ACI 设计原则

1. **换位思考**: 站在模型的角度, 仅凭工具描述和参数, 能否直观地知道如何使用这个工具? 如果你自己需要仔细思考, 模型也一样。
2. **完善文档**: 好的工具定义应包含使用示例、边界情况、输入格式要求、与其他工具的区别。
3. **迭代测试**: 在 Workbench 中用大量示例输入测试模型如何使用工具, 根据观察到的错误迭代优化。
4. **防错设计 (Poka-yoke)**: 修改参数设计使模型更难犯错。

7.2 工具格式的选择

同一个操作往往有多种描述方式。例如文件编辑可以用 diff 格式或重写整个文件; 结构化输出可以用 Markdown 或 JSON。虽然在软件工程中这些差异是表面的 (可以无损转换), 但对 LLM 来说, 不同格式的**难度差异巨大**。

设计维度	推荐做法	需要避免的做法
思考空间	给模型足够的 token 来「思考」再写入关键输出	要求模型在写出内容前就确定元信息 (如 diff 的行数)
格式自然度	使用模型在互联网文本中常见的格式	使用罕见或高度结构化的自定义格式
格式开销	最小化格式负担	要求精确计数千行代码或对代码做 JSON 转义
路径规范	使用绝对路径	使用相对路径 (容易因目录切换出错)

表 5: 工具格式设计的推荐做法与需要避免的做法

来自 SWE-bench 的实践经验

Anthropic 在构建 SWE-bench Agent 时, 在**工具优化上花费的时间远超整体 prompt 优化**。一个典型案例: 模型在使用相对路径的工具时频繁出错 (因为 Agent 会切换工作目录), 将工具改为强制要求绝对路径后, 模型使用完美无误。这说明好的 ACI 设计可以从根本上消除某类错误。

7.3 本章小结

工具设计是 Agent 成功的关键因素, 甚至比 prompt 优化更重要。ACI 的设计应遵循换位思考、完善文档、迭代测试、防错设计四大原则。工具格式应选择模型容易生成的自然格式, 避免不必要的格式开销。

8 总结与延伸

8.1 三大核心原则

Anthropic 在文章结尾提炼了构建 Agent 的三大核心原则：

构建 Agent 的三大核心原则

1. **保持设计简洁** (Maintain simplicity): Agent 的架构应该尽可能简单，只在证明有效的情况下增加复杂度。
2. **优先透明性** (Prioritize transparency): 明确展示 Agent 的规划步骤，让用户和开发者能理解 Agent 的决策过程。
3. **精心打造 ACI** (Carefully craft ACI): 通过完善的工具文档和测试，确保 Agent 能正确、高效地与外部世界交互。

8.2 从本文可以学到什么

这篇文章的价值不仅在于具体的设计模式，更在于其传达的**工程哲学**：

1. **从简到繁**：不要被「Agent」的概念所迷惑，先用最简单的方案尝试，再逐步升级。
2. **模式思维**：五种 Workflow 模式和 Agent 模式提供了一个实用的思考框架，帮助开发者在面对新任务时快速匹配合适的架构。
3. **工具即界面**：Agent 的能力上限很大程度上取决于工具设计的质量，而非模型本身或 prompt 的精巧程度。
4. **人机协作**：即使是最自主的 Agent，也应该设计人类监督和干预的机制。

8.3 拓展阅读

- Anthropic 官方 Cookbook: <https://github.com/anthropics/anthropic-cookbook> ——包含各种 agentic 模式的代码实现示例。
- Model Context Protocol (MCP): <https://modelcontextprotocol.io/> ——Agent 工具生态的统一标准。
- SWE-bench: <https://www.swebench.com/> ——评估编程 Agent 能力的标准基准。
- Claude Agent SDK: <https://github.com/anthropics/claude-agent-sdk> ——Anthropic 官方 Agent 开发工具包。